

Data Structures and Algorithms

Instructor: Dr. Hao Qin (秦浩)

E-mail: hao.qin@scupi.cn

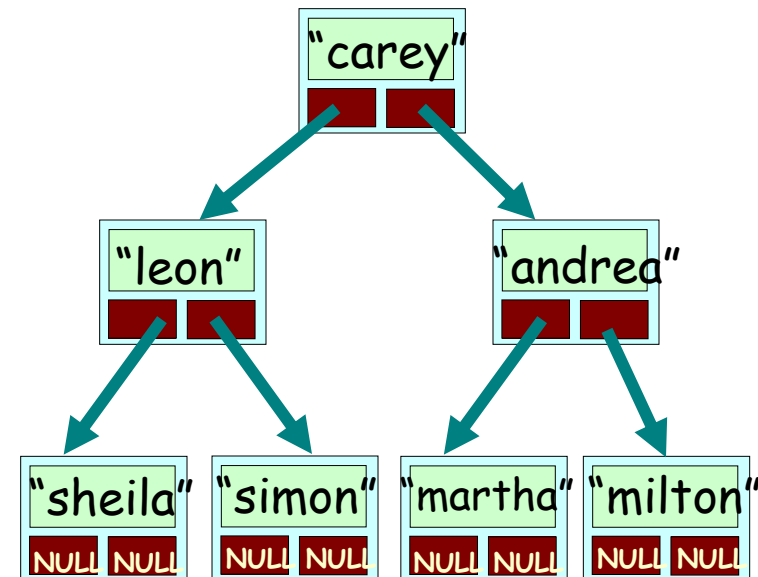
My Office: Room 514, SCUPI (North)

Trees

A **Tree** is a special **linked list-based data structure** that has many uses in Computer Science:

- To organize hierarchical data

A **Family** Tree

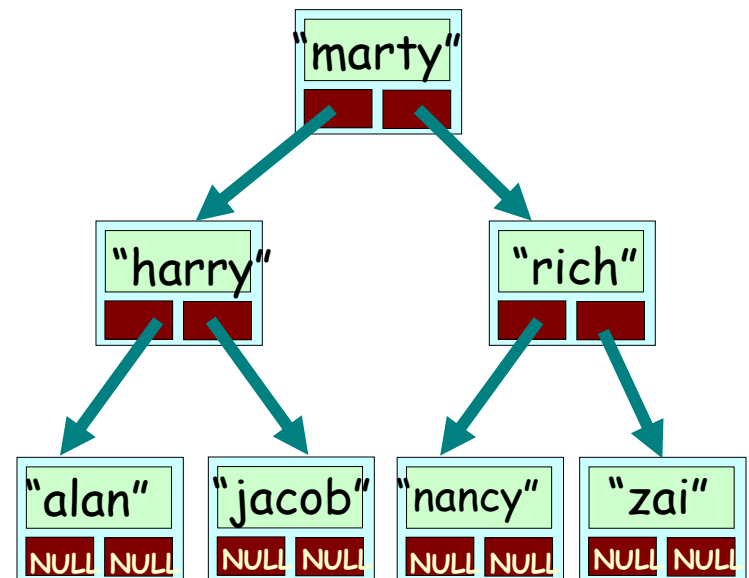


Trees

A **Tree** is a special **linked list-based data structure** that has many uses in Computer Science:

- To organize hierarchical data
- To make information easily searchable
-

A Binary **Search** Tree

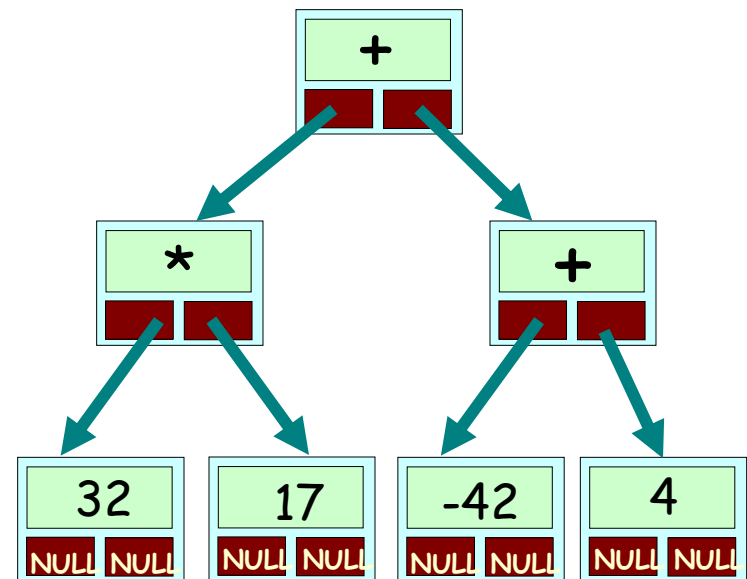


Trees

A **Tree** is a special **linked list-based data structure** that has many uses in Computer Science:

- To organize hierarchical data
- To make information easily searchable
- To simplify the evaluation of mathematical expressions

An **Expression** Tree

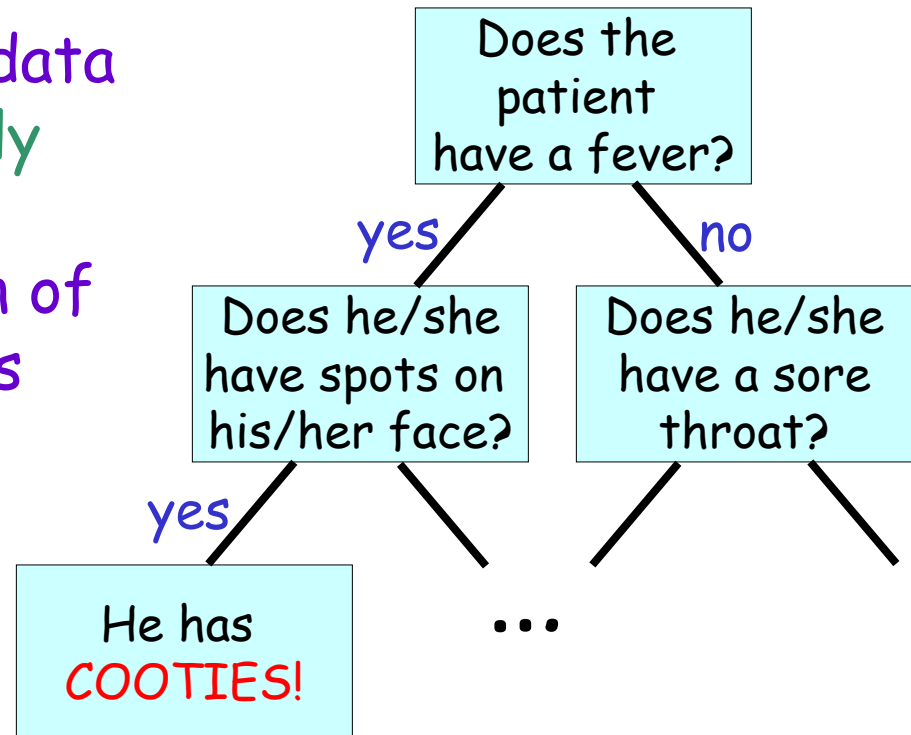


Trees

A **Tree** is a special **linked list-based data structure** that has many uses in Computer Science:

- To organize hierarchical data
- To make information easily searchable
- To simplify the evaluation of mathematical expressions
- To make decisions

A **Decision** Tree

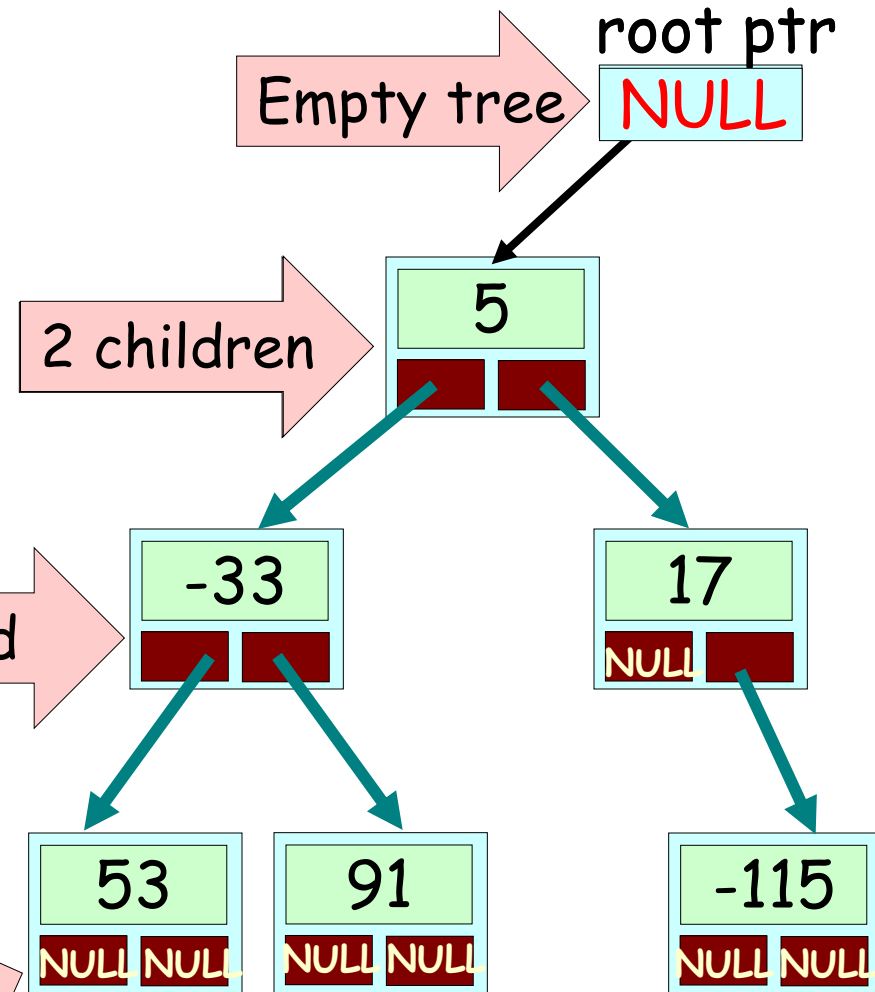


Basic Tree Facts

1. Trees are made of **nodes** (just like linked list nodes).
2. Every tree has a "**root**" pointer.
3. The top node of a tree is called its "**root**" node.
4. Every node may have zero or more "**children**" nodes.
5. A node with 0 children is called a "**leaf**" node.
6. A tree with no nodes is called an "**empty tree**."

```
struct node
{
    int value; // some value
    node *left, *right;
};

node *rootPtr;
```



Tree Nodes Can Have Many Children

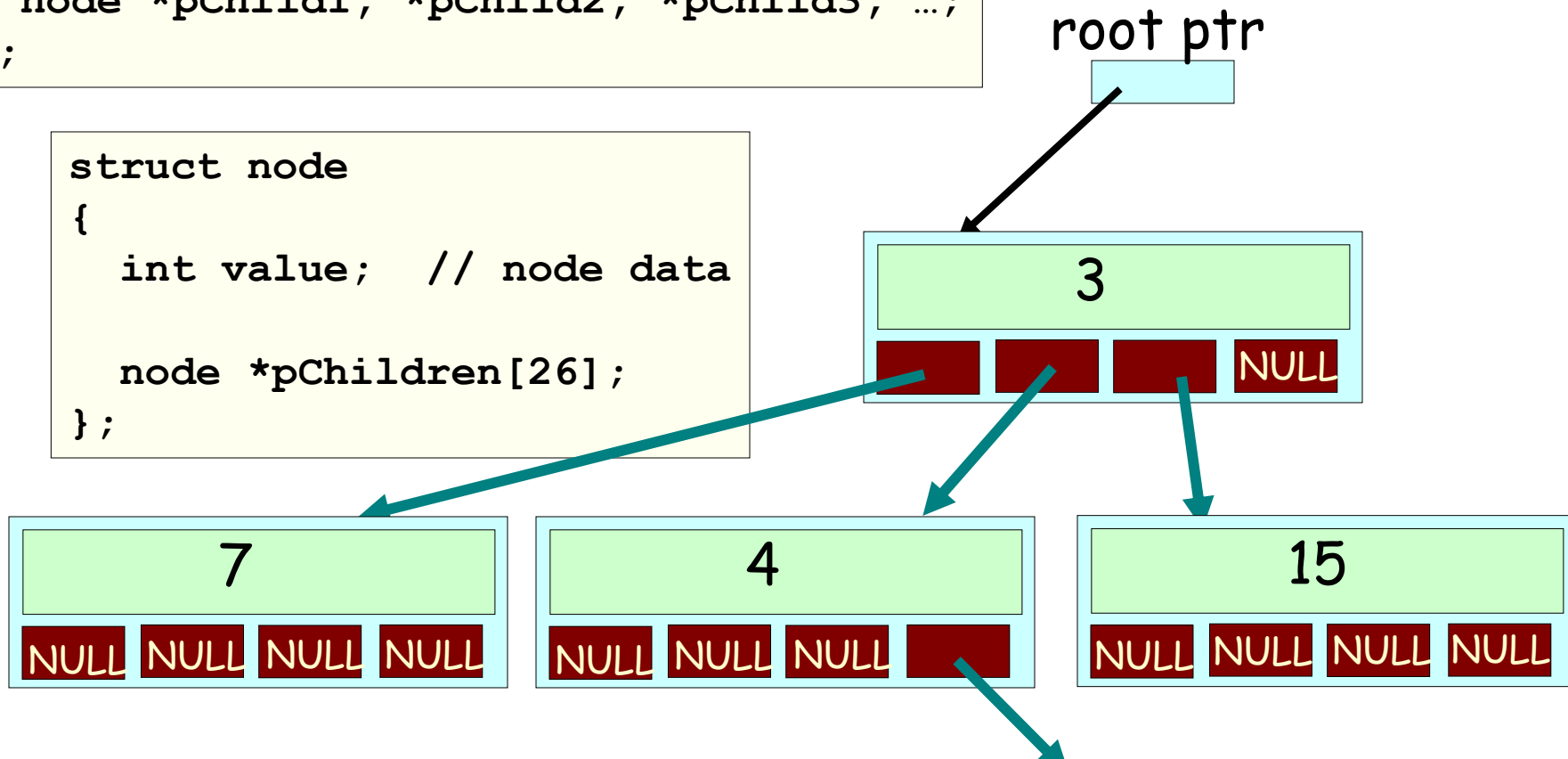
A tree node can have more than just two children:

```
struct node
{
    int value;    // node data

    node *pChild1, *pChild2, *pChild3, ...;
};
```

```
struct node
{
    int value;    // node data

    node *pChildren[26];
};
```



Binary Trees

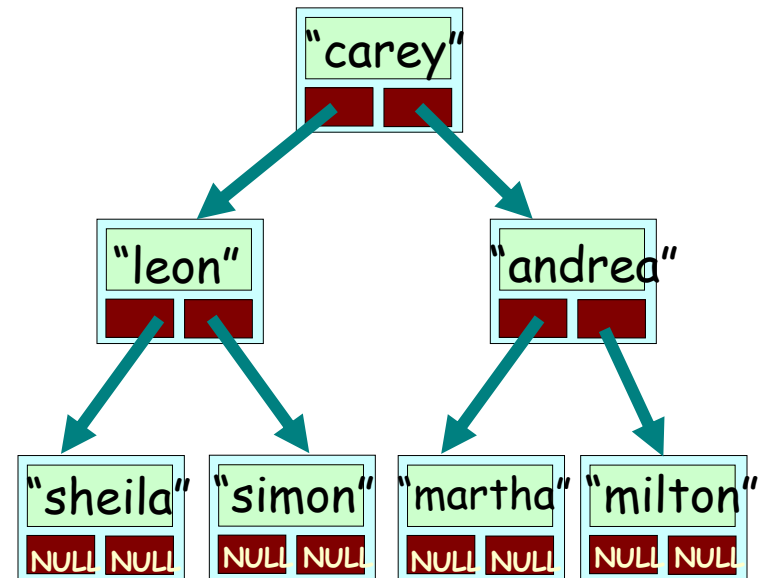
A **binary tree** is a special form of tree. In a binary tree, every node has at most **two children nodes**:

A left child and a right child.

```
struct BTNODE // binary tree node
{
    string value; // node data

    BTNODE *pLeft, *pRight;
};
```

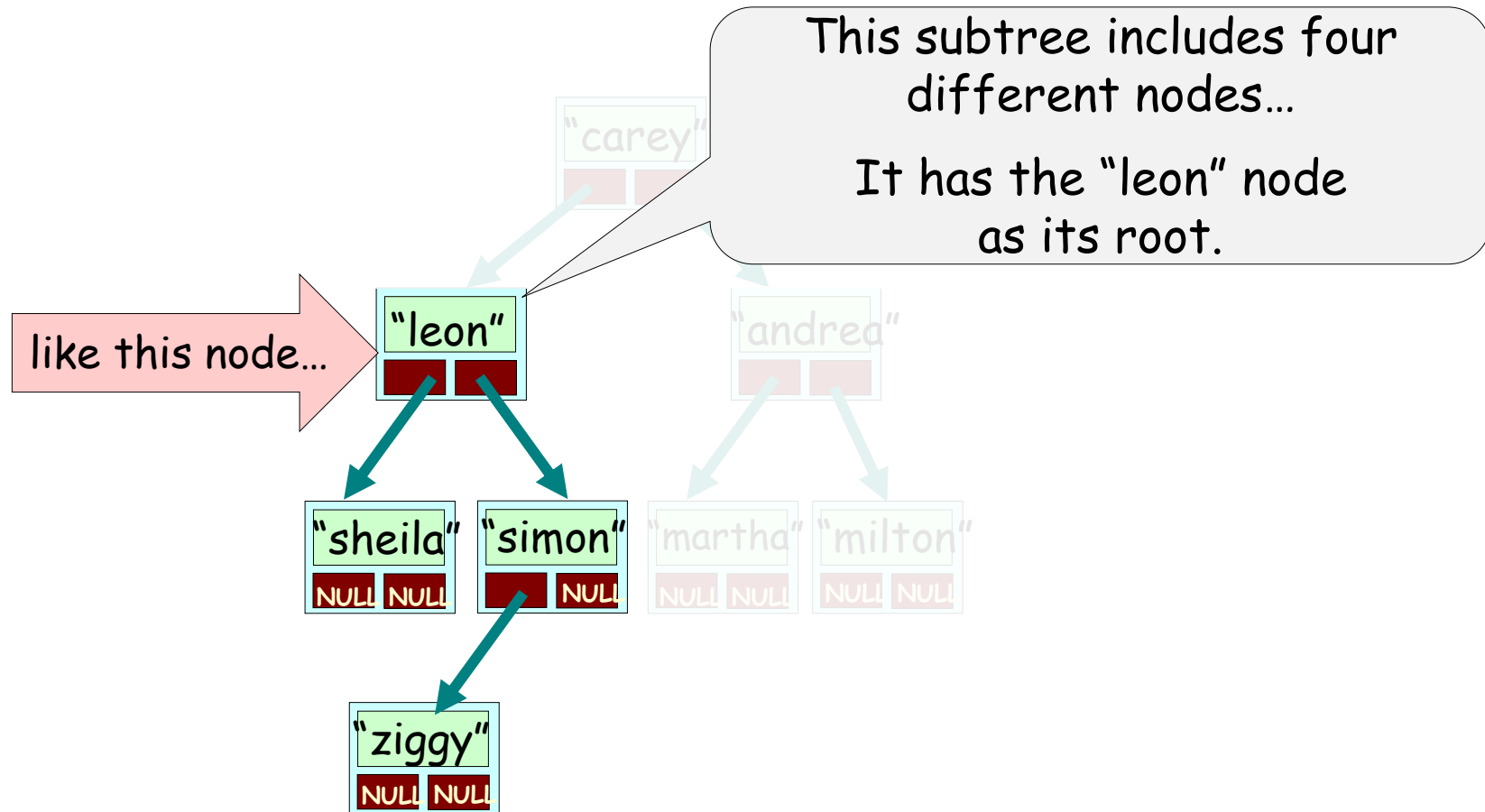
A **Binary** Tree



Binary Tree Subtrees

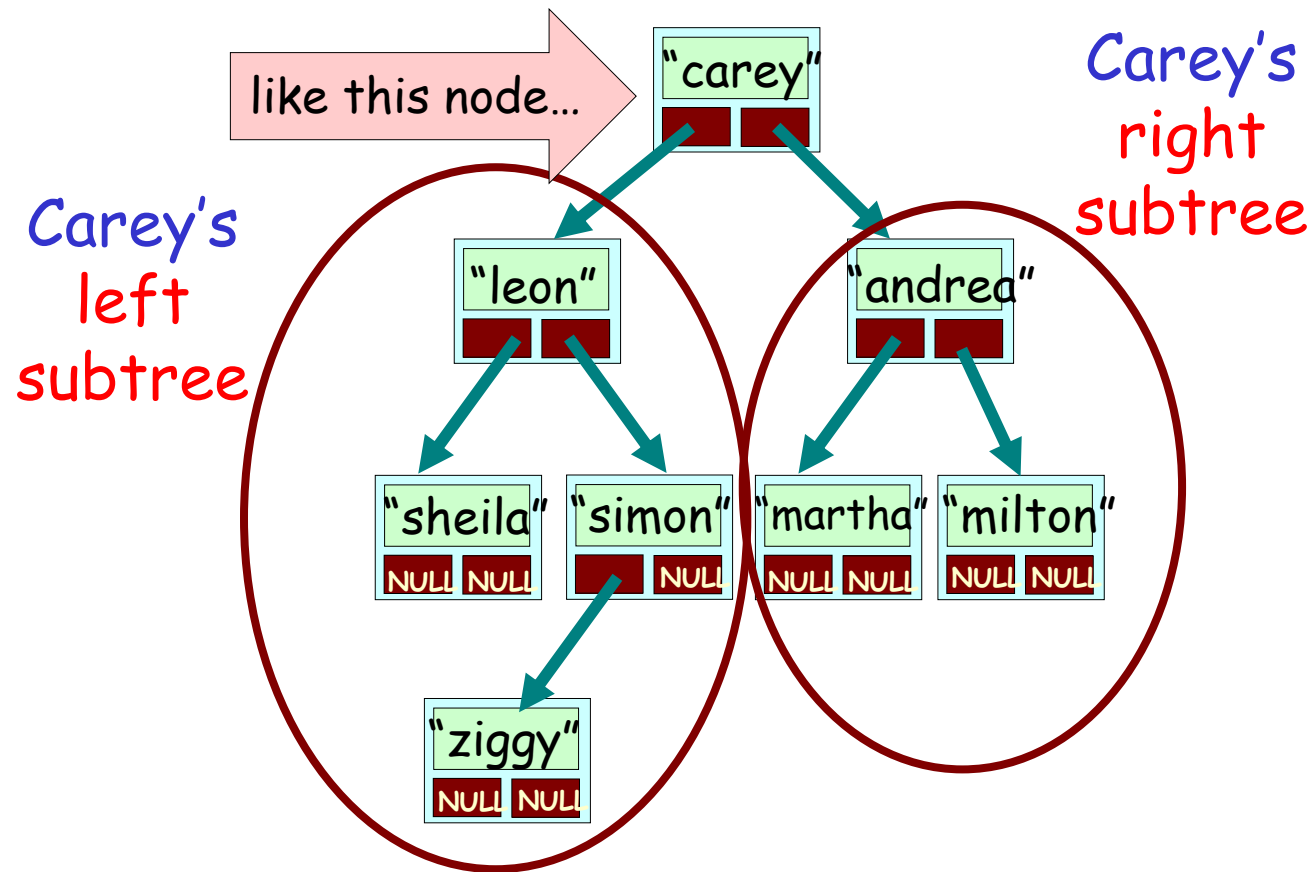
We can pick any node in the tree...

And then focus on its "**subtree**" - which includes it and all of nodes below it.



Binary Tree Subtrees

If we pick a node from our tree...
we can also identify its **left** and **right sub-trees**.



Operations on Binary Trees

The following are common operations that we might perform on a Binary Tree:

- enumerating all the items
- searching for an item
- adding a new item at a certain position on the tree
- deleting an item
- deleting the entire tree (destruction)
- removing a whole section of a tree (called **pruning**)
- adding a whole section to a tree (called **grafting**)

We'll learn about many of these operations over the next two classes.

```

struct BTNODE // node
{
    int value; // data
    BTNODE *left, *right;
};

```

```

main()
{
    → BTNODE *temp, *pRoot;

    → pRoot = new BTNODE;
    → pRoot->value = 5;

    → temp = new BTNODE;
    → temp->value = 7;
    → temp->left = NULL;
    → temp->right = NULL;
    → pRoot->left = temp;

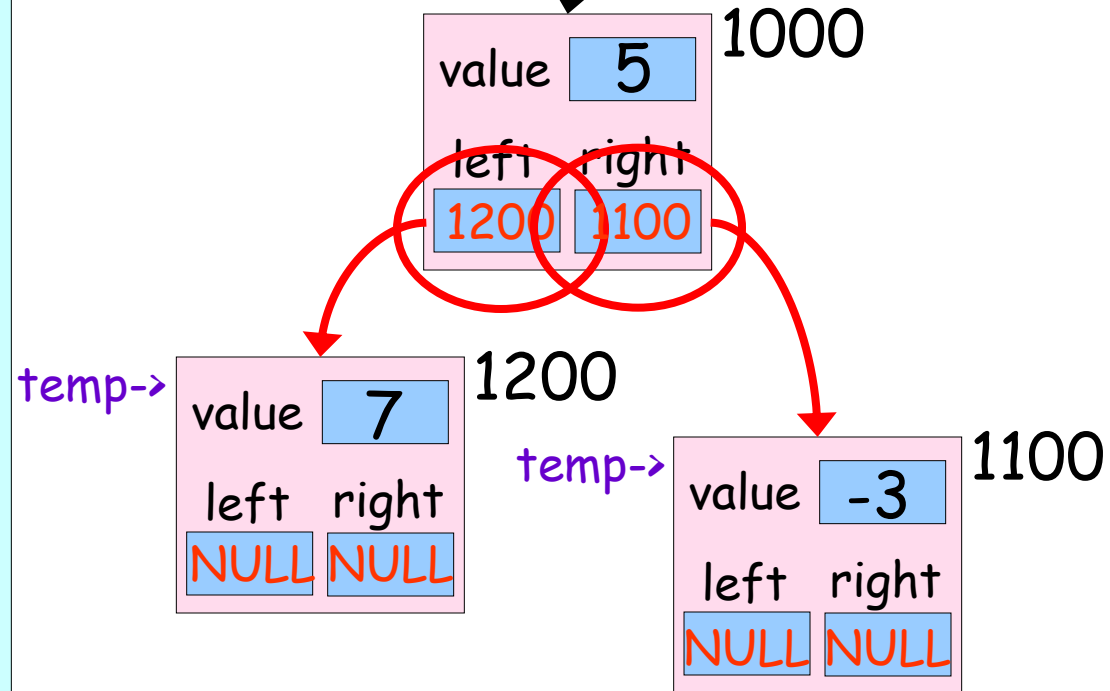
    → temp = new BTNODE;
    → temp->value = -3;
    → temp->left = NULL;
    → temp->right = NULL;
    → pRoot->right = temp;
    // etc...
}

```

A Simple Tree Example

temp **1200**

pRoot **1000**

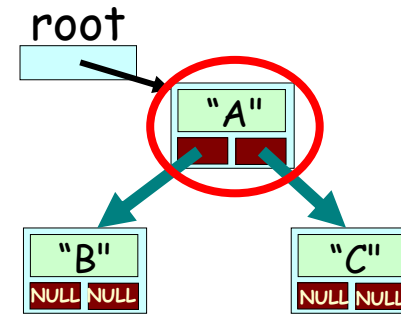


And of course, later we'd have to delete our tree's nodes.

Binary Tree Traversals

Just as we can traverse through linked lists, we can also traverse through the nodes of a tree.

All tree traversals start with the **root node**.



There are four common ways to traverse a tree - and each visits the tree's nodes in a different order:

Pre-order
Traversal

In-order
Traversal

Post-order
Traversal

These traversals all use **recursion**, and are nearly identical algorithms.

Level-order
Traversal

L.O.T. uses a **breadth-first-search** to visit nodes.

Pre-order, In-order and Post-order

Let's see the code for simple **pre-**, **in-**, and **post-order** traversals!

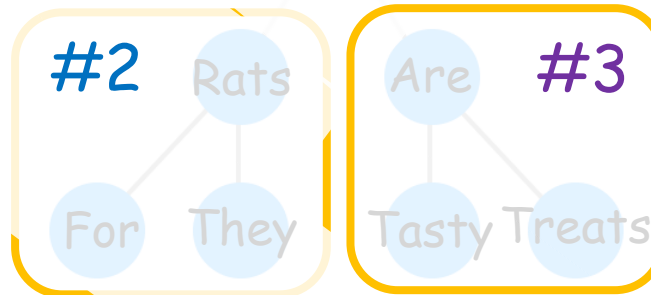
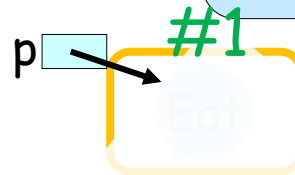
```
void preorder(Node *p)
{
    if (p == nullptr)
        return;
    cout << p->val << " ";
    preorder(p->left);
    preorder(p->right);
}
```

Finally, we fully **process** the **right subtree** of the current node using recursion.

First we
call the
left subtree

Then we **"process"** the
current node's value.

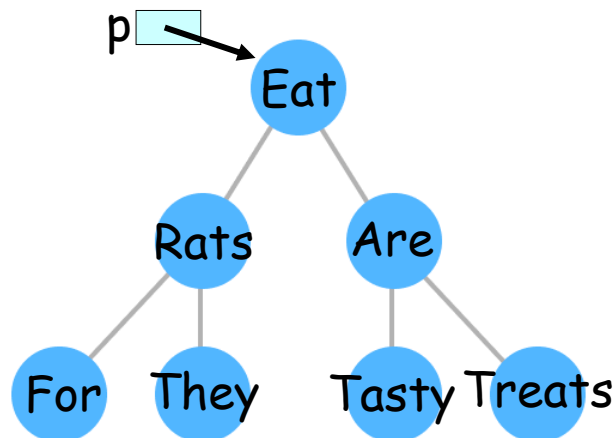
Then we **process** the
left subtree of the current
node using recursion.



Pre-order, In-order and Post-order

Let's see the code for simple **pre-**, **in-**, and **post-order** traversals!

```
void preorder(Node *p)
{
    if (p == nullptr)
        return;
    cout << p->value;
    preorder(p->left);
    preorder(p->right);
}
```



Pre-order, In-order and Post-order

Let's see the code for simple **pre-**, **in-**, and **post-order** traversals!

```
void preorder(Node *p)
{
    if (p == nullptr)
        return;
    cout << p->value;
    preorder(p->left);
    preorder(p->right);
}
```

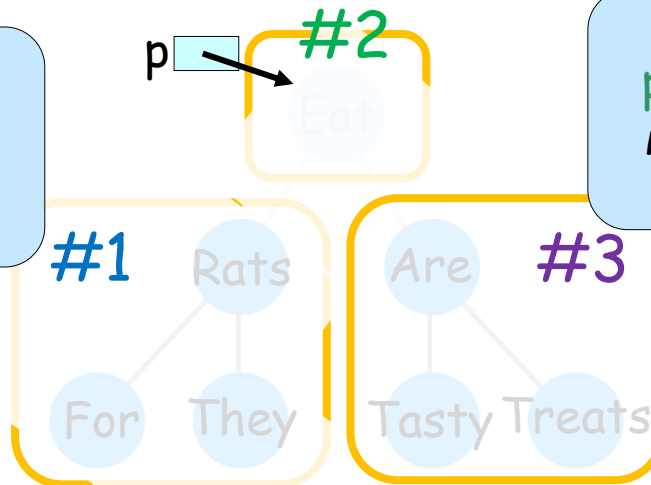
In pre-order, we **process** the current node first, then process its **left** and **right** subtrees.

```
void inorder(Node *p)
{
    if (p == nullptr)
        return;
    → inorder(p->left);
    → cout << p->value;
    → inorder(p->right);
}
```

Next let's see the in-order traversal!

How's it different from the pre-order

But with in-order, we **process** the current node *in between* processing its **left** and **right** subtrees.



Pre-order, In-order and Post-order

Let's see the code for simple **pre-**, **in-**, and **post-order** traversals!

```
void preorder(Node *p)
{
    if (p == nullptr)
        return;
    cout << p->value;
    preorder(p->left);
    preorder(p->right);
}
```

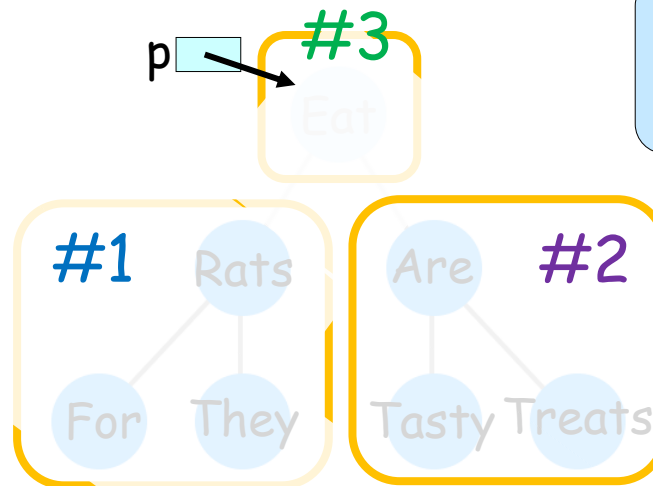
First we fully **process the left subtree** of the current node using recursion.

Then we fully **process the right subtree** of the current node using recursion.

```
void postorder(Node *p)
{
    if (p == nullptr)
        return;
    postorder(p->left);
    postorder(p->right);
    cout << p->value;
}
```

Finally, after everything else, we "**post-process**" the current node's value.

Finally, let's see the post-order traversal!



Pre-order, In-order and Post-order

Let's see the code for simple **pre-**, **in-**, and **post-order** traversals!

```
void preorder(Node *p)
{
    if (p == nullptr)
        return;
    cout << p->value;
    preorder(p->left);
    preorder(p->right);
}
```

```
void inorder(Node *p)
{
    if (p == nullptr)
        return;
    inorder(p->left);
    cout << p->value;
    inorder(p->right);
}
```

```
void postorder(Node *p)
{
    if (p == nullptr)
        return;
    postorder(p->left);
    postorder(p->right);
    cout << p->value;
}
```

Notice that the only difference is placement of the **processing step**!

But that small change results in major differences in the order the nodes are processed!

Ok, let's trace through our **pre-order** traversal!

The Pre-order Traversal

Output:

cur → nullptr

Our base case checks to see if **cur** points at an

Otherwise we **"process"** the value in the current node...

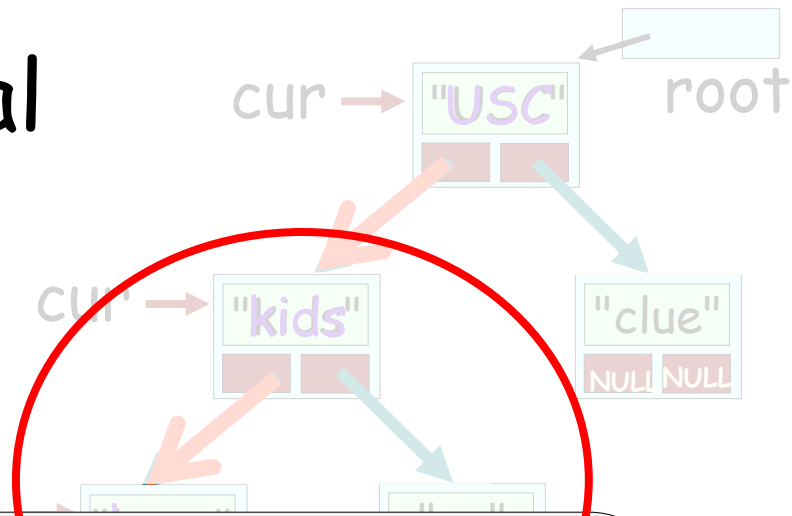
```
void preorder(Node *cur)
{
    if (cur == nullptr)
        return;

    cout << cur->value;

    preorder(cur->left); // Process nodes in left sub-tree.
    preorder(cur->right); // Process nodes in right sub-tree.
}

Node *root;

preorder(root);
```

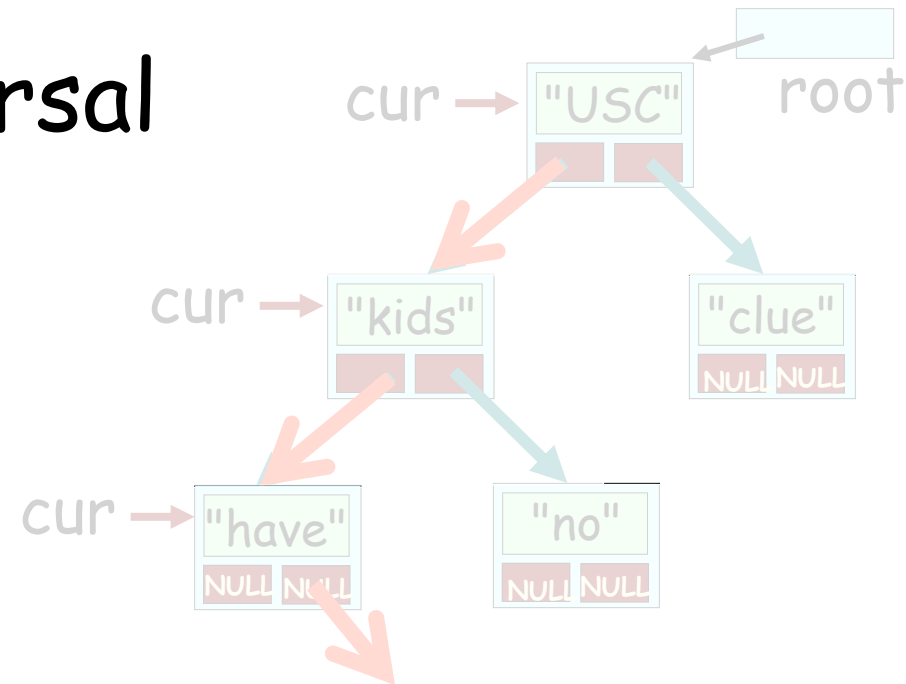


The Pre-order Traversal

Output:

USC kids have

cur → nullptr



```

void preorder(Node *cur)
{
    if (cur == nullptr)    // if empty, return...
        return;

    cout << cur->value;    // Process the current node.

    preorder(cur->left);    // Process nodes in left sub-tree.
    preorder(cur->right);   // Process nodes in right sub-tree.
}

```

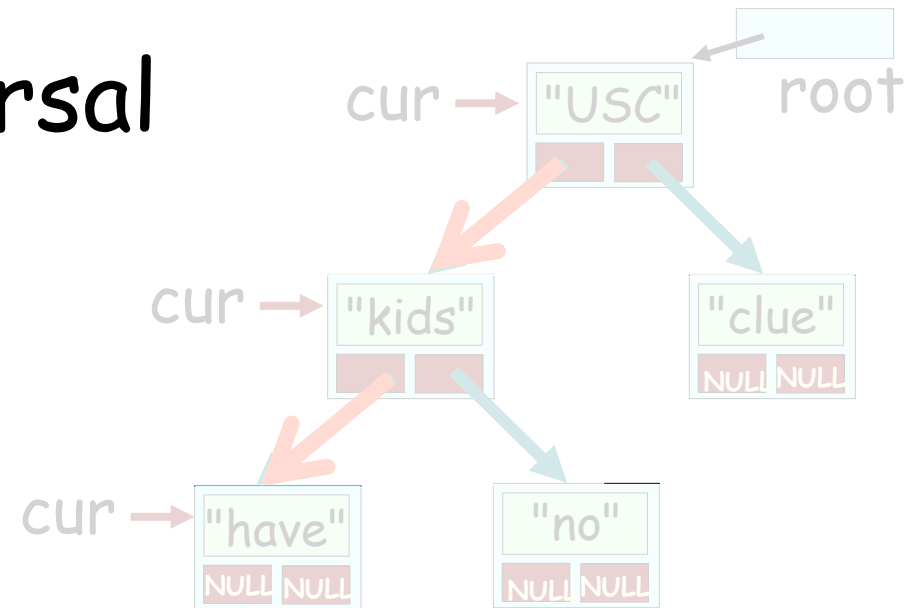
()
de *root;

preorder(root);

The Pre-order Traversal

Output:

USC kids have



```
void preorder(Node *cur)
```

```
{
{
{
if (cur == nullptr)      // if empty, return...
    return;

cout << cur->value;      // Process the current node.
preorder(cur->left);      // Process nodes in left sub-tree.
preorder(cur->right);     // Process nodes in right sub-tree.
}
}
}
```

```
n()
```

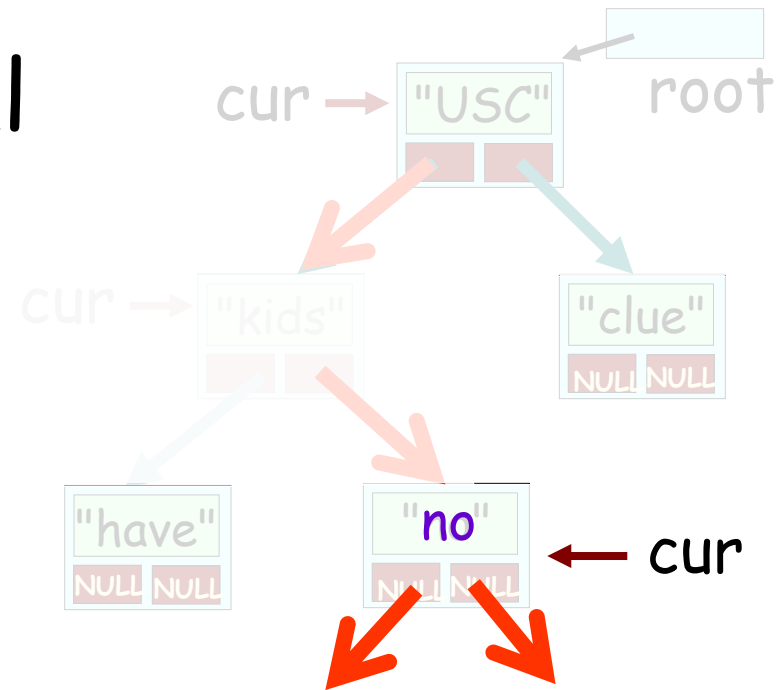
```
Node *root;
```

```
preorder(root);
```

The Pre-order Traversal

Output:

USC kids have



```

void preorder(Node *cur)
{
    if (cur == nullptr)    // if empty, return...
        return;

    cout << cur->value;    // Process the current node.

    preorder(cur->left);    // Process nodes in left sub-tree.
    preorder(cur->right);   // Process nodes in right sub-tree.
}

```

```

Node *root;

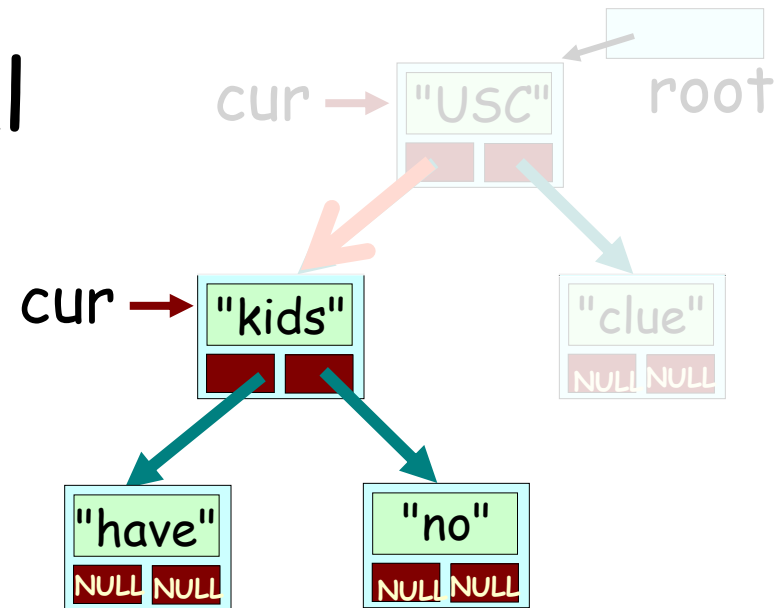
preorder(root);

```

The Pre-order Traversal

Output:

USC kids have no



```
void preorder(Node *cur)
```

```
{
{   if (cur == nullptr)        // if empty, return...
    return;
```

```
    cout << cur->value;        // Process the current node.
```

```
    preorder(cur->left);        // Process nodes in left sub-tree.
```

```
    preorder(cur->right);       // Process nodes in right sub-tree.
```

```
}
```

```
}
```

```
main()
```

```
Node *root;
```

```
...
```

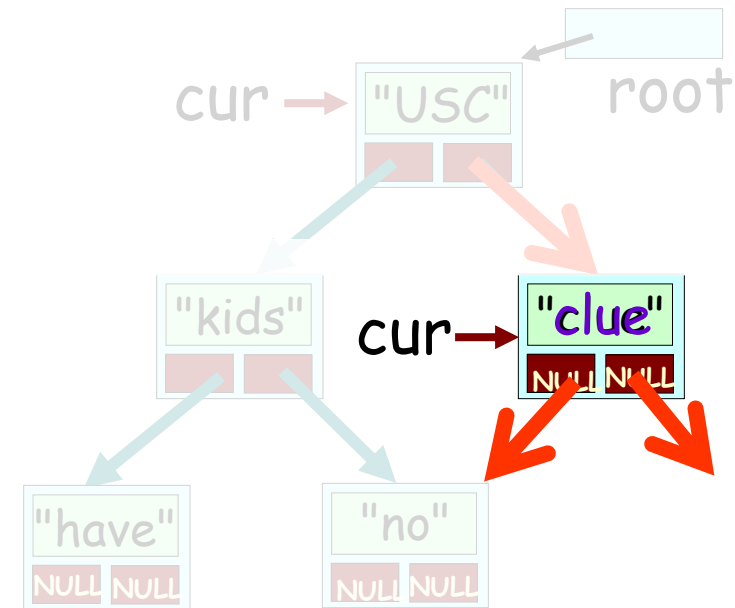
```
preorder(root);
```

```
}
```

The Pre-order Traversal

Output:

USC kids have no



```

void preorder(Node *cur)
{
    if (cur == nullptr)    // if empty, return...
        return;

    cout << cur->value;    // Process the current node.

    preorder(cur->left);    // Process nodes in left sub-tree.
    preorder(cur->right);   // Process nodes in right sub-tree.
}

```

```

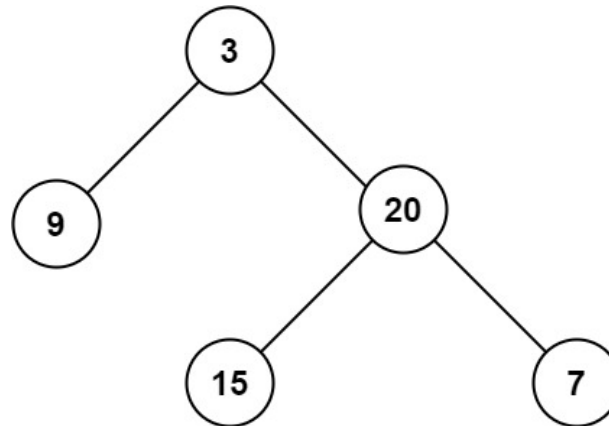
main()
{
    Node *root;
    ...
    preorder(root);
}

```


Practice1: Maximum Depth of Binary Tree

Given a binary tree, find its maximum depth. The maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example:



Input: root = [3,9,20,null,null,15,7] **Output:** 3

Solution of Practice1

```
#include <iostream>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(nullptr),
right(nullptr) {}
};

int maxDepth(TreeNode* root) {
    if (root == nullptr) return 0;
    int leftDepth = maxDepth(root->left);
    int rightDepth = maxDepth(root->right);
    return 1 + max(leftDepth, rightDepth);
}
```

```
int main() {
    TreeNode* root = new
TreeNode(3);
    root->left = new TreeNode(9);
    root->right = new
TreeNode(20);
    root->right->left = new
TreeNode(15);
    root->right->right = new
TreeNode(7);

    cout << "Maximum Depth: " <<
maxDepth(root) << endl;
    return 0;
}
```

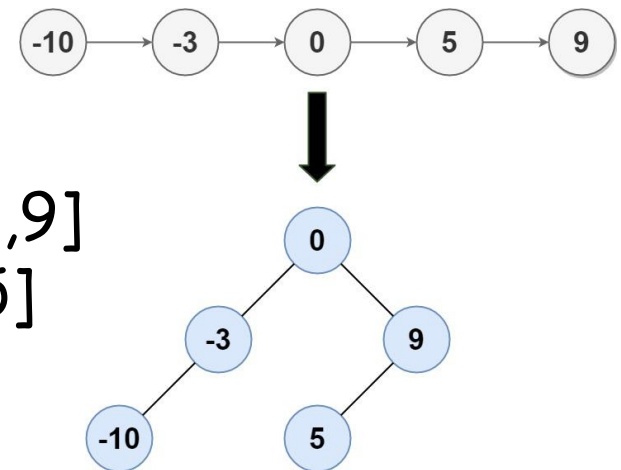
Practice2: Convert Sorted List to Binary Search Tree

Given the head of a singly linked list where elements are sorted in ascending order, convert it to a height-balanced binary search tree (BST). A height-balanced binary tree is defined as a binary tree in which the depth of the two subtrees of every node never differ by more than one.

Example:

Input: head = [-10,-3,0,5,9]

Output: [0,-3,9,-10,null,5]



```
#include <iostream>
using namespace std;
```

Solution of Practice2

```
struct ListNode {
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(nullptr) {}
};

struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};
```

```
ListNode* findMiddle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    ListNode* prev = nullptr;
    while (fast && fast->next) {
        prev = slow;
        slow = slow->next;
        fast = fast->next->next;
    }
    if (prev) {
        prev->next = nullptr;
    }
    return slow;
}
```

```
TreeNode* sortedListToBST(ListNode* head) {
    if (!head) return nullptr;
    ListNode* middle = findMiddle(head);
    TreeNode* root = new TreeNode(middle->val);
    if (head == middle) return root;
    root->left = sortedListToBST(head);
    root->right = sortedListToBST(middle->next);
    return root;
}
```

```
void printTree(TreeNode* root) {
    if (!root) return;
    cout << root->val << " ";
    printTree(root->left);
    printTree(root->right);
}
```

```
int main() {
    ListNode* head = new ListNode(-10);
    head->next = new ListNode(-3);
    head->next->next = new ListNode(0);
    head->next->next->next = new ListNode(5);
    head->next->next->next->next = new ListNode(9);
```

```
    TreeNode* root = sortedListToBST(head);
    printTree(root);
    cout << endl;
```

```
    // Clean up allocated nodes
    delete head->next->next->next->next;
    delete head->next->next->next;
    delete head->next->next;
    delete head->next;
    delete head;
```

```
    return 0;
}
```